



STOCKFLOW

Master Project Documentation — v1.0 Final

☒ **This document contains all 13 sections including the new AI Tools section**
Sections 1–12 are unchanged from the previous version. Section 13 is new.

HOW TO USE

Upload this file OR paste Section 12 prompt into any new Claude, ChatGPT, or Copilot chat.
Claude will instantly understand the full project. Section 13 explains exactly how to use each AI tool.

Contents

- 1. Project Overview** — What StockFlow is, who it is for, example search result
- 2. System Architecture** — Modular Monolith decision, module structure, request flow
- 3. Optional Features System** — Feature flags, warehouse optional, feature toggle screen
- 4. Database Design** — All 15 tables, flexible attribute system, key table specs
- 5. Product Structure** — Category→Model→Variant hierarchy, bulk creation, pricing
- 6. User Roles & Permissions** — Four roles, full permission matrix
- 7. Setup Wizard** — 9-step wizard, warehouse yes/no question, blank slate
- 8. Screens & Interface** — All screens, Admin Panel sub-sections, button rules
- 9. Tech Stack & Packaging** — Tools, pipeline, database location, launcher script
- 10. Folder Structure & Roadmap** — Complete folder tree, 14-phase build order, future phases
- 11. All Key Decisions** — Complete decision log — 20 decisions with reasons
- 12. Prompt for New Chats** — Copy-paste block to resume project in any AI chat
- 13. Using AI Tools to Build StockFlow** — GitHub Copilot, Claude, ChatGPT — setup and usage guide ←
NEW

13. Using AI Tools to Build StockFlow

You do not need to write every line of code from scratch. Three AI tools can dramatically speed up development: GitHub Copilot, Claude (Anthropic), and ChatGPT. Here is exactly how to use each one effectively for this project.

13.1 The Three Tools and What Each Does Best

Tool	Best For	Free?	Where to Get It
GitHub Copilot	Writing code inside VS Code as you type — autocompletes functions, generates boilerplate, fills in repetitive patterns	Free tier available	github.com/copilot
Claude	Designing architecture, explaining concepts, writing entire files on request, reviewing and fixing code, building documentation	Free tier + paid	claude.ai
ChatGPT	Quick questions, debugging errors, explaining Django concepts, generating SQL queries, alternative explanations	Free tier + paid	chatgpt.com

You do not need all three. GitHub Copilot + Claude is the strongest combination for this project. Use ChatGPT as a backup when you want a second opinion.

13.2 GitHub Copilot — Setup and Usage

Setting It Up

- Go to github.com/copilot and sign in with your GitHub account
- Free tier: available for students and open source contributors — apply at education.github.com
- Paid tier: \$10/month for individuals if not eligible for free
- Install VS Code (code.visualstudio.com) — this is your code editor
- Inside VS Code: open Extensions panel → search 'GitHub Copilot' → Install
- Sign in to GitHub when prompted — Copilot activates automatically

How It Works in Your Project

Copilot watches what you are typing and suggests completions. You press Tab to accept a suggestion or keep typing to ignore it. For StockFlow, it is most useful for:

EXAMPLE 1 — You start typing a Django model:

```
class ProductVariation(models.Model):
    model = models.ForeignKey(          ← Copilot suggests the rest

Copilot completes: 'ProductModel, on_delete=models.PROTECT)'
```

EXAMPLE 2 — You write a comment describing what you need:

```
# Function to get total stock for a variation across all warehouses

Copilot writes the entire function below your comment.
```

EXAMPLE 3 — You start a Django view:

```
def search_stock(request):
    # Get category, model and attribute filters from request

    Copilot writes the filter logic, queryset, and return statement.
```

Best Copilot Practices for StockFlow

- Write a comment above every function before you write the function — Copilot reads the comment and generates better code
- Type the function name clearly — `search_stock`, `transfer_stock`, `get_price_history` — Copilot understands the intent
- Accept suggestions line by line — do not blindly accept whole blocks without reading them
- Use it for repetitive patterns — if you write one Django model it will generate similar ones very accurately
- Great for writing HTML forms and Tailwind CSS layouts — just describe what you need in a comment

13.3 Claude — Setup and Usage

Setting It Up

- Go to claude.ai and create a free account
- Free tier is enough to start — paid plan (Claude Pro) gives more messages per day
- No installation needed — it runs in your browser
- To continue the StockFlow project in any new chat: upload this document OR paste the prompt from Section 12

How to Use Claude for StockFlow

Claude is best used for larger tasks — designing whole files, explaining decisions, reviewing code, and fixing bugs. Here are the exact types of prompts that work well:

Task	What to Ask Claude
Generate a full <code>models.py</code> file	"Here is my StockFlow project context [paste Section 12]. Write the complete <code>models.py</code> file for the inventory module with all 8 tables."
Write a Django view	"Write the <code>search_stock</code> view for StockFlow. It should filter by category, model, and any attribute values selected from dropdowns and return per-warehouse quantities."
Fix a bug	"This Django view is throwing an error: [paste error message and code]. Fix it and explain what was wrong."
Write HTML template	"Write the search screen HTML template using Tailwind CSS. It should have dropdown filters at the top and a results table below showing warehouse name and quantity."
Explain a concept	"Explain how Django atomic transactions work and show me exactly how to use them for the stock transfer feature in StockFlow."
Review code	"Review this Django view for StockFlow and tell me if there are any bugs, security issues, or better ways to write it: [paste code]"
Write the installer config	"Write the Inno Setup <code>.iss</code> script for StockFlow. App installs to Program Files, creates desktop icon, starts on Windows boot."
Generate test cases	"Write Django unit tests for the stock transfer feature — test that stock is deducted correctly, rollback works on failure, and negative stock is prevented."

The Golden Rule for Claude

```
Always start every new chat with:

1. Upload this document OR paste the prompt from Section 12
2. Then immediately say what you want built

Example:
'I am building StockFlow [context from Section 12].
Now write the complete models.py for the inventory module.'
```

Without the context, Claude does not know about StockFlow and will give generic Django answers instead of project-specific code.

13.4 ChatGPT — Setup and Usage

Setting It Up

- Go to chatgpt.com and create a free account
- Free tier uses GPT-4o mini — sufficient for most tasks
- ChatGPT Plus (\$20/month) uses GPT-4o — better for complex code
- Same approach as Claude — paste context from Section 12 at the start of each chat

When to Use ChatGPT vs Claude

Use Claude When	Use ChatGPT When
Writing entire files or modules	Quick one-off questions about Django or Python
Architecture decisions	Debugging a specific error message
Reviewing and improving existing code	Explaining what a piece of code does
This project specifically (it knows it)	Getting a second opinion on something Claude said
Building documentation	Finding the right Python library for a task

13.5 Recommended Workflow — Day to Day

Here is the most efficient way to use all three tools together when building StockFlow:

```
STEP 1 — Plan the feature (Claude)
Ask: 'Explain how I should structure the transfer screen view
and what database queries it needs.'
Claude gives you the plan and any code patterns to follow.

STEP 2 — Write the code (VS Code + Copilot)
Open VS Code. Create the file. Start typing.
Copilot autocompletes as you write — accept what looks right.
Write comments to guide Copilot when you need specific logic.

STEP 3 — Fix errors (Claude or ChatGPT)
If something does not work, paste the error + the relevant code.
Ask: 'This Django view is throwing [error]. Here is the code: [paste].
Fix it and explain what was wrong.'
```

STEP 4 — Review (Claude)

After finishing a feature, paste the code and ask Claude to review it.

Ask: 'Review this for bugs, security issues, or improvements.'

13.6 How to Write Good Prompts for Code Generation

The quality of the code you get depends entirely on how clearly you describe what you need. Here is the pattern that works best:

GOOD PROMPT STRUCTURE:

1. Context → 'I am building StockFlow [paste Section 12 summary]'
2. What → 'Write the Django view for the stock transfer screen'
3. Requirements → 'It must: validate quantity, use atomic transaction, prevent negative stock, log to stock_transfer table'
4. Tech → 'Use Django 4.2, Class-Based View, return JSON response'
5. Example → 'The transfer form sends: variation_id, from_warehouse_id, to_warehouse_id, quantity'

WEAK PROMPT (gets generic code):
'Write a stock transfer view'

STRONG PROMPT (gets StockFlow-specific code):
'I am building StockFlow [context]. Write the transfer_stock Django view. It must check the from_warehouse has enough quantity, wrap the deduct and add operations in a transaction.atomic() block, rollback if either fails, create a stock_transfer and transfer_line record, and return a JSON response with success or error message. Use function-based view.'

13.7 GitHub — Setting Up the Repository

Since StockFlow is open source, you need a GitHub repository. Here is the full setup:

Creating the Repository

- Go to github.com and sign in
- Click New Repository
- Name it: stockflow
- Set visibility to Public (required for open source)
- Add a README, .gitignore (Python), and MIT License
- Click Create Repository

Connecting Your Local Code

```
# In VS Code terminal, inside your stockflow/ folder:

git init
git add .
git commit -m 'Initial commit - StockFlow project structure'
git branch -M main
git remote add origin https://github.com/[your-username]/stockflow.git
git push -u origin main
```

What to Put in .gitignore

```
# Never commit these to GitHub:
db.sqlite3      ← contains real shop data
*.pyc
```

```

__pycache__/  

.env           ← contains secret keys  

dist/         ← PyInstaller output  

build/  

*.spec

```

Publishing a Release (the setup.exe)

- Build the installer using PyInstaller + Inno Setup
- Go to your GitHub repository → Releases → Create a new release
- Tag it as v1.0.0
- Upload StockFlow_Setup.exe as an asset
- Write release notes describing what is included
- Publish — users can now download directly from GitHub

13.8 Quick Reference — Which Tool for Which Task

Task	Best Tool	Notes
Write Django models.py	Claude	Paste Section 12 context, ask for full file
Autocomplete as you type	GitHub Copilot	Works automatically in VS Code
Fix a Django error	Claude or ChatGPT	Paste error message + relevant code
Write HTML + Tailwind templates	Claude + Copilot	Claude designs it, Copilot helps as you type
Write PyInstaller config	Claude	Ask for stockflow.spec file specifically
Write Inno Setup script	Claude	Ask for setup.iss with auto-start on boot
Understand a Django concept	ChatGPT	Good for quick explanations
Review finished code	Claude	Ask for bugs, security, improvements
Write unit tests	Claude	Ask for Django TestCase for each feature
Git commands	ChatGPT	Quick reference for git operations
Write README for GitHub	Claude	Ask Claude to write a professional README
Debug Tailwind CSS layout	ChatGPT	Quick CSS questions work well

13.9 Important Rules When Using AI-Generated Code

AI tools generate code that looks correct but may have subtle bugs. Always follow these rules:

- Always read and understand every line of code before using it — never blindly copy-paste
- Test every feature after adding AI-generated code — do not wait until the end
- If the generated code does something unexpected, paste it back and ask the AI to explain it line by line
- Never paste real customer data or passwords into Claude, ChatGPT, or Copilot
- If AI-generated code conflicts with the StockFlow design decisions in this document — the document wins
- Use AI to write code faster, not to replace understanding — you need to know what your code does

